

Interfaces in Java: Entkopplung und Testbarkeit am Beispiel eines Zahlungssystems und eines Loggers

Benjamin Malicke ©

6. März 2026

Inhaltsverzeichnis

1	Was sind Interfaces in Java und wozu nutzt man sie?	2
1.1	Interface vs. Klasse	2
1.2	Warum Interfaces? Typische Einsatzgründe	2
1.3	Interfaces vs. abstrakte Klassen	3
1.3.1	Gemeinsamkeiten	3
1.3.2	Unterschiede	3
1.3.3	Wann nimmt man was?	4
1.3.4	Mehrfachvererbung und das sogenannte Diamond-Problem	4
1.4	Überleitung zum Zahlungssystem-Beispiel	4
2	Ziel	5
3	Das Interface <code>PaymentService</code>	5
4	Implementierungen des Interfaces	5
4.1	Kreditkartenzahlung	5
4.2	PayPal-Zahlung	6
4.3	Kauf auf Rechnung	6
5	Shop-Klasse	7
6	<code>FakePaymentService</code> für Tests	7
7	Main-Klasse: Zusammenspiel der Komponenten	7
8	UML-Klassendiagramm des Zahlungssystems	8
9	Weitere Anwendung von Interfaces: Logging mit <code>Logger</code>	9
9.1	Das Interface <code>Logger</code>	9
9.2	Implementierung 1: Konsolen-Logger	9
9.3	Implementierung 2: Datei-Logger	10
9.4	Verwendung des Loggers im Anwendungscode	10
9.5	Laufzeitentscheidung: Konsole oder Datei?	11
9.6	Vorteile des <code>Logger</code> -Interfaces	11
9.7	Testen mit einem einfachen <code>FakeLogger</code>	11
10	UML-Klassendiagramm des <code>Logger</code>-Beispiels	12
11	Zusammenfassung	13
12	Anhang: Prinzip „Programmiere gegen ein Interface, nicht gegen eine Implementierung“	14

1 Was sind Interfaces in Java und wozu nutzt man sie?

In Java ist ein *Interface* ein spezieller Typ, der festlegt, *welche* Methoden eine Klasse bereitstellen muss, ohne vorzugeben, *wie* diese Methoden intern umgesetzt werden. Man kann sich ein Interface wie einen „Vertrag“ oder eine Schnittstellen-Beschreibung vorstellen:

- Es definiert **Methodenköpfe** (Signaturen), aber keine Implementierung (also keinen Methodenrumpf mit Code).
- Klassen, die ein Interface **implementieren**, müssen alle darin deklarierten Methoden bereitstellen.
- Ab Java 8 können Interfaces auch **default**-Methoden (mit Standardimplementierung) und **static**-Methoden enthalten.

Der große Vorteil von Interfaces ist, dass sie den Code entkoppeln: Der aufrufende Code kennt nur das Interface (also den Vertrag), nicht aber die konkrete Implementierung dahinter. Dadurch wird Software flexibler, übersichtlicher und besser testbar.

1.1 Interface vs. Klasse

Eine normale Klasse beschreibt sowohl *Daten* (Attribute) als auch *Verhalten* (Methoden) und kann direkt instanziiert werden:

- **Klasse:** Enthält Felder, Konstruktoren und Methoden mit Implementierung. Man kann Objekte dieser Klasse erzeugen.
- **Interface:** Enthält in der Regel nur öffentliche, abstrakte Methodendeklarationen (plus evtl. Konstanten und default-Methoden). Es kann nicht direkt instanziiert werden, sondern muss von einer Klasse **implementiert** werden.

In Java gibt es nur einfache Vererbung zwischen Klassen (eine Klasse kann nur von genau einer anderen Klasse **extends** erben), aber eine Klasse darf **mehrere Interfaces** implementieren. Dadurch kann eine Klasse mehrere „Rollen“ oder „Fähigkeiten“ annehmen.

1.2 Warum Interfaces? Typische Einsatzgründe

Interfaces werden in der Praxis aus mehreren Gründen sehr häufig eingesetzt:

1. Einheitliche Schnittstelle (Vertrag)

Unterschiedliche Klassen können dieselben Methoden anbieten.

Beispiel: In der Java-Standardbibliothek gibt es das Interface **List**. Verschiedene Klassen wie **ArrayList** oder **LinkedList** implementieren dieses Interface. Aus Sicht des aufrufenden Codes spielt es keine Rolle, welche konkrete Implementierung verwendet wird, solange das **List**-Interface erfüllt ist.

2. Polymorphie und Entkopplung

Ein häufiger Grundsatz in der Softwareentwicklung lautet: „Programmiere gegen ein Interface, nicht gegen eine Implementierung.“

Statt sich an eine konkrete Klasse zu binden, verwendet man überall den Interface-Typ. Welche konkrete Implementierung tatsächlich benutzt wird, kann dann leicht ausgetauscht werden (z. B. für unterschiedliche Datenbanken oder verschiedene Zahlungsarten).

3. Mehrfachvererbung von Fähigkeiten

Da eine Klasse mehrere Interfaces implementieren kann, kann sie verschiedene Fähigkeiten gleichzeitig besitzen, z. B. **Runnable** (in einem Thread ausführbar) und **Serializable** (seriellisierbar für Speicherung oder Übertragung).

4. Saubere Architektur und Schichten

In größeren Anwendungen trennt man verschiedene Schichten (z. B. Präsentationsschicht, Geschäftslogik, Datenzugriff). Oft definiert man pro Schicht oder Komponente ein Interface (z. B. `UserRepository`), das von unterschiedlichen Implementierungen erfüllt werden kann (z. B. Zugriff auf eine echte Datenbank, auf eine Testdatenbank oder auf einen In-Memory-Speicher).

5. Bessere Testbarkeit (Mocks und Fakes)

Wenn Klassen nur noch gegen Interfaces arbeiten, kann man diese in Tests sehr leicht durch „Fake“- oder „Mock“- Implementierungen austauschen. So lassen sich einzelne Komponenten isoliert testen, ohne zum Beispiel eine echte Datenbank oder einen echten Zahlungsanbieter zu benötigen.

1.3 Interfaces vs. abstrakte Klassen

Neben Interfaces gibt es in Java auch *abstrakte Klassen*. Beide Konzepte dienen dazu, Gemeinsamkeiten zwischen Klassen zu beschreiben und Verhalten vorzugeben, das Unterklassen bzw. Implementierungen bereitstellen müssen. Trotzdem unterscheiden sie sich in einigen wichtigen Punkten.

1.3.1 Gemeinsamkeiten

Sowohl Interfaces als auch abstrakte Klassen

- können **abstrakte Methoden** definieren, die von Unterklassen bzw. implementierenden Klassen überschrieben werden müssen,
- können als **gemeinsamer Typ** verwendet werden, über den man polymorph arbeitet (z. B. eine Variable vom Typ des Interfaces oder der abstrakten Klasse),
- dienen dazu, **Struktur und Vertrag** für mehrere Klassen festzulegen.

1.3.2 Unterschiede

Trotz der Ähnlichkeiten gibt es klare Unterschiede in der Verwendung und in den Möglichkeiten:

- **Vererbung vs. Implementierung**
Eine Klasse kann in Java nur von *einer* anderen Klasse erben (einschließlich einer abstrakten Klasse), aber sie kann **mehrere Interfaces** gleichzeitig implementieren.
Folge: Interfaces sind ideal, um mehrere „Fähigkeiten“ oder Rollen zu kombinieren (z. B. `Runnable`, `Serializable`), während eine abstrakte Klasse eher eine gemeinsame Basisklasse für eine Hierarchie darstellt.
- **Gemeinsamer Code (Implementierung)**
Abstrakte Klassen können neben abstrakten Methoden auch **fertige Implementierungen** enthalten, inklusive Felder, Konstruktoren und nicht-abstrakten Methoden.
Interfaces definieren in erster Linie nur das „*was*“ (Methoden- Signaturen). Ab Java 8 können zwar auch Interfaces **default**- und **static**-Methoden mit Implementierung besitzen, sie enthalten aber in der Regel keine Zustandsdaten (Instanzfelder), die von allen Implementierungen gemeinsam genutzt werden.
Faustregel: Wenn Sie gemeinsamen Zustand und umfangreiche gemeinsame Logik teilen möchten, ist eine abstrakte Klasse oft die passendere Wahl.
- **Zustand (Felder)**
Abstrakte Klassen können Instanzvariablen besitzen, die von den Unterklassen geerbt werden. So kann gemeinsamer Zustand zentral in der abstrakten Basisklasse verwaltet werden.
Interfaces enthalten typischerweise nur **Konstanten** (`public static final`) und keine veränderlichen Instanzfelder.

- **Design-Perspektive**

Interface: Beschreibt hauptsächlich ein *Verhalten* oder eine *Fähigkeit* (z. B. “kann bezahlt werden“, “kann sortiert werden“, “ist vergleichbar“). Es geht um das “*was*“.

Abstrakte Klasse: Beschreibt eher einen *gemeinsamen Basistyp* innerhalb einer Vererbungshierarchie mit gemeinsamem Code und ggf. gemeinsamem Zustand (z. B. `AbstractVehicle` für unterschiedliche Fahrzeugarten). Es geht stärker um das “*was*“ und “*wie*“.

- **Einsatz in bestehenden Hierarchien**

Wenn bereits eine Klassenhierarchie existiert oder eine Klasse schon von einer anderen Klasse erben muss, ist es oft nicht mehr möglich, noch zusätzlich eine abstrakte Basisklasse einzuziehen, da Mehrfachvererbung von Klassen in Java nicht erlaubt ist.

Durch Interfaces können Sie jedoch zusätzliche Verträge und Fähigkeiten hinzufügen, ohne die bestehende Vererbung zu verändern.

1.3.3 Wann nimmt man was?

Eine grobe Orientierung für die Praxis könnte so aussehen:

- Verwenden Sie ein **Interface**, wenn Sie ein *gemeinsames Verhalten* definieren wollen, das von vielen, möglicherweise sehr unterschiedlichen Klassen umgesetzt werden soll, und wenn Mehrfachimplementierung hilfreich ist.
- Verwenden Sie eine **abstrakte Klasse**, wenn Sie eine *gemeinsame Basis* mit geteilter Implementierung und ggf. gemeinsamem Zustand für eine eng verwandte Gruppe von Klassen schaffen möchten.

1.3.4 Mehrfachvererbung und das sogenannte Diamond-Problem

In einigen Programmiersprachen ist sogenannte **Mehrfachvererbung von Klassen** erlaubt, also dass eine Klasse gleichzeitig von mehreren Oberklassen erbt. Dabei kann das sogenannte *Diamond-Problem* auftreten: Zwei unterschiedliche Oberklassen erben von derselben Basisklasse, und eine Unterklasse erbt wiederum von beiden Oberklassen. Dann ist oft unklar, welche geerbte Implementierung an einer bestimmten Stelle gelten soll.

Java vermeidet dieses Problem bewusst, indem es **keine Mehrfachvererbung von Klassen** erlaubt (eine Klasse kann nur eine Oberklasse haben). Mehrfachvererbung ist in Java nur über Interfaces möglich. Da Interfaces in der Regel keinen gemeinsamen Instanzzustand teilen, sind Konflikte deutlich seltener und leichter aufzulösen.

Seit Java 8 können allerdings auch Interfaces **default**-Methoden mit Implementierung enthalten. Falls mehrere implementierte Interfaces für dieselbe Methode eine unterschiedliche **default**-Implementierung anbieten, muss die Klasse diesen Konflikt explizit auflösen, indem sie die Methode überschreibt und sich für eine konkrete Implementierung entscheidet.

1.4 Überleitung zum Zahlungssystem-Beispiel

Im weiteren Verlauf dieses Dokuments wird gezeigt, wie ein Interface `PaymentService` für ein einfaches Shopsystem definiert werden kann. Dieses Interface beschreibt die gemeinsame Fähigkeit unterschiedlicher Zahlungsarten, einen Betrag zu bezahlen. Konkrete Klassen wie `CreditCardPaymentService`, `PaypalPaymentService` und `InvoicePaymentService` implementieren dieses Interface jeweils auf ihre eigene Weise.

Der Vorteil: Der Shop selbst arbeitet nur mit dem Typ `PaymentService` und ist damit unabhängig von der konkreten Zahlungsart. Dadurch können neue Zahlungsmethoden später leicht hinzugefügt oder bestehende Implementierungen ersetzt werden, ohne dass der Rest des Codes angepasst werden muss.

2 Ziel

Dieses Dokument zeigt, wie man in Java ein *Interface* nutzt, um verschiedene Zahlungsarten (z. B. Kreditkarte, PayPal, Rechnung) über eine vereinheitlichte Schnittstelle `PaymentService` anzusprechen.

Zusätzlich wird geprüft, ob übergebene Werte sinnvoll sind (zum Beispiel, ob der Betrag `amount` größer als 0 ist). Wenn die Werte ungültig sind, wird eine Fehlermeldung ausgegeben und die Methode liefert `false` zurück, ohne eine Exception zu werfen.

3 Das Interface PaymentService

```
1 public interface PaymentService {
2
3     /**
4      * Versucht, einen bestimmten Betrag zu bezahlen.
5      *
6      * @param amount Betrag in Euro; muss > 0 sein
7      * @return true, wenn Zahlung erfolgreich, sonst false
8      */
9     boolean pay(double amount);
10 }
```

4 Implementierungen des Interfaces

Im Folgenden werden drei unterschiedliche Implementierungen vorgestellt:

- `CreditCardPaymentService` (Kreditkarte),
- `PaypalPaymentService` (PayPal),
- `InvoicePaymentService` (Kauf auf Rechnung).

Alle Implementierungen

1. prüfen, ob `amount` größer als 0 ist,
2. geben bei ungültigem Betrag eine Fehlermeldung aus,
3. liefern in diesem Fall den Wert `false` zurück.

4.1 Kreditkartenzahlung

```
1 public class CreditCardPaymentService implements PaymentService {
2
3     private String cardNumber;
4
5     public CreditCardPaymentService(String cardNumber) {
6         this.cardNumber = cardNumber;
7     }
8
9     @Override
10    public boolean pay(double amount) {
11        if (amount <= 0) {
12            System.out.println("Fehler: Betrag muss groesser als 0 sein."
13                + " Angegebener Betrag: " + amount + " EUR");
14            return false;
15        }
16    }
```

```

16
17 System.out.println("Zahle " + amount + " EUR mit Kreditkarte "
18 + cardNumber + ".");
19 // Hier könnte ein echter Aufruf zum Kreditkarten-Provider stehen
20 // Wir vereinfachen und tun so, als ob die Zahlung immer klappt.
21 return true;
22 }
23 }

```

4.2 PayPal-Zahlung

```

1 public class PaypalPaymentService implements PaymentService {
2
3     private String email;
4
5     public PaypalPaymentService(String email) {
6         this.email = email;
7     }
8
9     @Override
10    public boolean pay(double amount) {
11        if (amount <= 0) {
12            System.out.println("Fehler: Betrag muss groesser als 0 sein."
13 + " Angegebener Betrag: " + amount + " EUR");
14            return false;
15        }
16
17        System.out.println("Zahle " + amount + " EUR mit PayPal-Konto "
18 + email + ".");
19        // Hier könnte ein echter Aufruf zur PayPal-API stehen.
20        return true;
21    }
22 }

```

4.3 Kauf auf Rechnung

```

1 public class InvoicePaymentService implements PaymentService {
2
3     private String customerId;
4
5     public InvoicePaymentService(String customerId) {
6         this.customerId = customerId;
7     }
8
9     @Override
10    public boolean pay(double amount) {
11        if (amount <= 0) {
12            System.out.println("Fehler: Betrag muss groesser als 0 sein."
13 + " Angegebener Betrag: " + amount + " EUR");
14            return false;
15        }
16
17        System.out.println("Erstelle Rechnung ueber " + amount
18 + " EUR fuer Kunden-ID " + customerId + ".");
19        // Hier könnte eine Datenbank-Operation zur Rechnungserstellung stehen.
20        return true;
21    }
22 }

```

5 Shop-Klasse

Die Shop-Klasse kennt nur das Interface `PaymentService` und ist damit unabhängig von der konkreten Zahlungsart.

```
1 public class Shop {
2
3     private final PaymentService paymentService;
4
5     public Shop(PaymentService paymentService) {
6         this.paymentService = paymentService;
7     }
8
9     public void checkout(double amount) {
10        System.out.println("Starte Checkout mit Betrag: " + amount + " EUR");
11
12        boolean success = paymentService.pay(amount);
13        if (success) {
14            System.out.println("Bestellung erfolgreich bezahlt!\n");
15        } else {
16            System.out.println("Zahlung fehlgeschlagen. Bitte pruefen Sie"
17                + " Ihre Eingaben.\n");
18        }
19    }
20 }
```

6 FakePaymentService für Tests

Mit einem einfachen Fake lässt sich das Verhalten des Shops testen, ohne wirklich eine Zahlung auszuführen.

```
1 public class FakePaymentService implements PaymentService {
2
3     private boolean result;
4
5     public FakePaymentService(boolean result) {
6         this.result = result;
7     }
8
9     @Override
10    public boolean pay(double amount) {
11        if (amount <= 0) {
12            System.out.println("FakePaymentService: ungeltiger Betrag: "
13                + amount + " EUR");
14            return false;
15        }
16        System.out.println("FakePaymentService: simuliere Zahlung von "
17            + amount + " EUR.");
18        return result;
19    }
20 }
```

7 Main-Klasse: Zusammenspiel der Komponenten

Die Main-Klasse zeigt, wie leicht sich mit Interfaces unterschiedliche Zahlungsarten austauschen lassen. Außerdem wird das Verhalten mit gültigen und ungültigen Beträgen demonstriert.

```
1 public class Main {
```

```

2
3 public static void main(String[] args) {
4 // Beispiel 1: Kreditkartenzahlung mit gueltigem Betrag
5 PaymentService creditCardPayment =
6 new CreditCardPaymentService("1234-5678-9012-3456");
7 Shop shop1 = new Shop(creditCardPayment);
8 shop1.checkout(49.99);
9
10 // Beispiel 2: PayPal-Zahlung mit ungueltigem Betrag (0 oder negativ)
11 PaymentService paypalPayment =
12 new PaypalPaymentService("kunde@example.com");
13 Shop shop2 = new Shop(paypalPayment);
14 shop2.checkout(0.0); // sollte eine Fehlermeldung liefern
15 shop2.checkout(-10.0); // ebenfalls Fehlermeldung
16
17 // Beispiel 3: Kauf auf Rechnung mit gueltigem Betrag
18 PaymentService invoicePayment =
19 new InvoicePaymentService("KUNDE_123");
20 Shop shop3 = new Shop(invoicePayment);
21 shop3.checkout(99.90);
22
23 // Beispiel 4: Einsatz des FakePaymentService in Tests
24 PaymentService fakePaymentSuccess = new FakePaymentService(true);
25 Shop testShopSuccess = new Shop(fakePaymentSuccess);
26 testShopSuccess.checkout(10.0);
27
28 PaymentService fakePaymentFail = new FakePaymentService(false);
29 Shop testShopFail = new Shop(fakePaymentFail);
30 testShopFail.checkout(10.0);
31 }
32 }

```

8 UML-Klassendiagramm des Zahlungssystems

Abbildung 1 zeigt das UML-Klassendiagramm für das Beispiel mit dem Interface `PaymentService`, den konkreten Zahlungsarten und der `Shop`-Klasse.

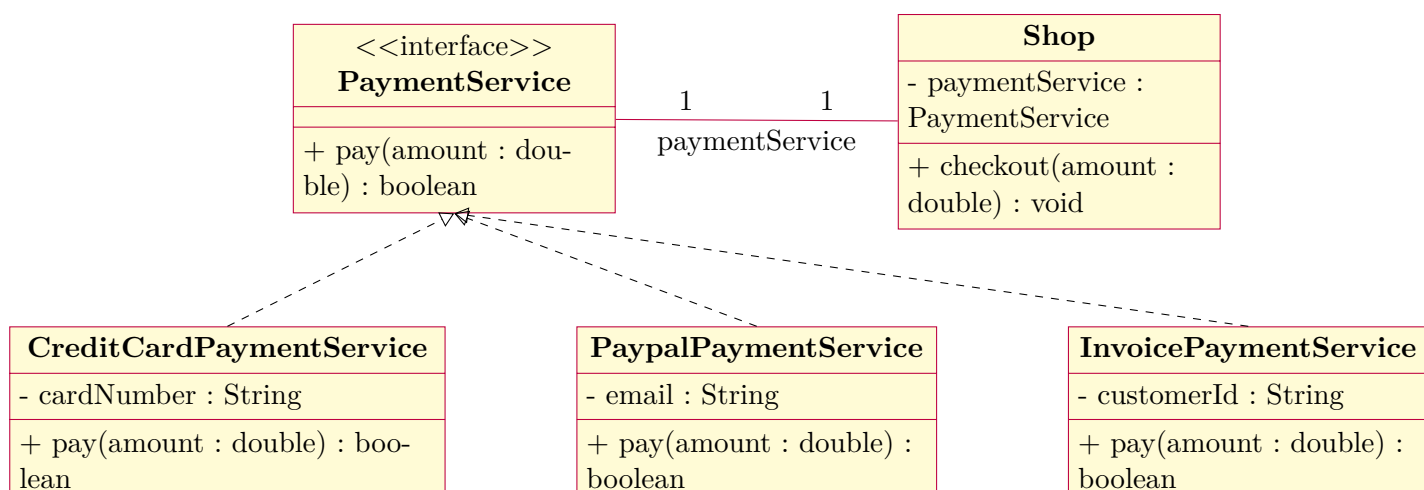


Abbildung 1: UML-Klassendiagramm des Zahlungssystems mit Interface `PaymentService`.

9 Weitere Anwendung von Interfaces: Logging mit Logger

Ein weiteres typisches Einsatzgebiet für Interfaces ist das *Logging*. Oft soll eine Anwendung Meldungen über ihren Ablauf ausgeben, zum Beispiel:

- zur Fehlersuche (Debugging),
- zur Überwachung im Betrieb,
- zum Nachvollziehen von Abläufen (Auditing).

Dabei ist es aber häufig egal, *wohin* geloggt wird (Konsole, Datei, Netzwerk, Datenbank etc.). Genau hier hilft ein Interface: Es legt fest, *welche* Log-Methoden vorhanden sein müssen, ohne sich auf eine konkrete Ausgabeart festzulegen.

9.1 Das Interface Logger

Zuerst definieren wir ein Interface `Logger`, das drei Methoden für unterschiedliche Loglevel bereitstellt:

```
1 public interface Logger {
2
3     void info(String message);
4
5     void warn(String message);
6
7     void error(String message);
8 }
```

Jede Klasse, die dieses Interface implementiert, verpflichtet sich dazu, diese Methoden anzubieten. Wie genau die Meldungen ausgegeben werden, bleibt der jeweiligen Implementierung überlassen.

9.2 Implementierung 1: Konsolen-Logger

Eine einfache Implementierung schreibt die Log-Nachrichten direkt auf die Konsole:

```
1 public class ConsoleLogger implements Logger {
2
3     @Override
4     public void info(String message) {
5         System.out.println("INFO: " + message);
6     }
7
8     @Override
9     public void warn(String message) {
10        System.out.println("WARN: " + message);
11    }
12
13    @Override
14    public void error(String message) {
15        System.err.println("ERROR: " + message);
16    }
17 }
```

Hier wird `System.out` für `info` und `warn` sowie `System.err` für Fehlermeldungen verwendet. Die Klasse erfüllt den Vertrag des Interfaces, ohne dass der aufrufende Code wissen muss, dass die Ausgabe auf der Konsole erfolgt.

9.3 Implementierung 2: Datei-Logger

Als zweite Implementierung schreiben wir die Log-Meldungen in eine Datei. Dies ist ein häufiges Muster in echten Anwendungen (hier stark vereinfacht):

```
1  import java.io.FileWriter;
2  import java.io.IOException;
3  import java.time.LocalDateTime;
4
5  public class FileLogger implements Logger {
6
7      private String fileName;
8
9      public FileLogger(String fileName) {
10         this.fileName = fileName;
11     }
12
13     private void writeLine(String level, String message) {
14         try (FileWriter writer = new FileWriter(fileName, true)) {
15             writer.write(LocalDateTime.now() + " [" + level + "] "
16                 + message + System.lineSeparator());
17         } catch (IOException e) {
18             System.err.println("Konnte nicht in Logdatei schreiben: "
19                 + e.getMessage());
20         }
21     }
22
23     @Override
24     public void info(String message) {
25         writeLine("INFO", message);
26     }
27
28     @Override
29     public void warn(String message) {
30         writeLine("WARN", message);
31     }
32
33     @Override
34     public void error(String message) {
35         writeLine("ERROR", message);
36     }
37 }
```

Die Methode `writeLine` kapselt hier die eigentliche Schreiblogik und wird von `info`, `warn` und `error` mit unterschiedlichen Leveln aufgerufen.

9.4 Verwendung des Loggers im Anwendungscode

Der große Vorteil der Arbeit mit einem Interface zeigt sich in der Verwendung: Die Klasse `OrderService` kennt nur den Typ `Logger`, nicht aber die konkrete Implementierung dahinter.

```
1  public class OrderService {
2
3      private final Logger logger;
4
5      public OrderService(Logger logger) {
6          this.logger = logger;
7      }
8
9      public void placeOrder(String productId, int quantity) {
10         logger.info("Starte Bestellung: Produkt=" + productId
```

```

11     + ", Menge=" + quantity);
12
13     if (quantity <= 0) {
14         logger.warn("Ungueltige Menge: " + quantity);
15         return;
16     }
17
18     // Hier würde die eigentliche Bestelllogik stehen...
19
20     logger.info("Bestellung erfolgreich abgeschlossen.");
21 }
22 }

```

Die Klasse `OrderService` ruft nur die Methoden des Interfaces auf. Ob dabei tatsächlich ein `ConsoleLogger`, ein `FileLogger` oder eine ganz andere Implementierung verwendet wird, spielt für den Code keine Rolle.

9.5 Laufzeitentscheidung: Konsole oder Datei?

In einer `Main`-Klasse kann nun zur Laufzeit entschieden werden, welche konkrete Logger-Implementierung verwendet werden soll:

```

1  public class Main {
2
3      public static void main(String[] args) {
4          // Variante 1: Logging auf die Konsole
5          Logger consoleLogger = new ConsoleLogger();
6          OrderService service1 = new OrderService(consoleLogger);
7          service1.placeOrder("ABC-123", 2);
8
9          // Variante 2: Logging in eine Datei
10         Logger fileLogger = new FileLogger("app.log");
11         OrderService service2 = new OrderService(fileLogger);
12         service2.placeOrder("XYZ-999", -1);
13     }
14 }

```

Ohne eine Zeile in `OrderService` zu ändern, kann das Verhalten des Loggings komplett umgestellt werden. In größeren Projekten würde man diese Entscheidung typischerweise über Konfigurationsdateien oder Dependency Injection treffen.

9.6 Vorteile des Logger-Interfaces

Dieses Beispiel macht mehrere Vorteile von Interfaces deutlich:

- **Austauschbarkeit:** Neue Logger-Implementierungen (z. B. Datenbank- oder Netzwerk-Logger) können hinzugefügt werden, ohne die bestehenden Klassen wie `OrderService` zu verändern.
- **Entkopplung:** Der Anwendungscode ist nicht an eine konkrete Technik zur Ausgabe gebunden (Konsole vs. Datei), sondern kennt nur den Vertrag (Interface).
- **Testbarkeit:** In Tests kann man leicht einen einfachen „FakeLogger“ einsetzen, der die Meldungen nur sammelt oder ignoriert, statt tatsächlich Dateien zu beschreiben.

9.7 Testen mit einem einfachen FakeLogger

Für automatisierte Tests ist es oft unpraktisch, echte Logger zu verwenden, zum Beispiel wenn sie Dateien beschreiben oder auf externe Systeme zugreifen. Stattdessen kann man eine sehr einfache Implementierung des Interfaces nutzen, die nur intern merkt, was geloggt wurde. Solch ein „FakeLogger“ ist leicht auszuwerten und verursacht keine Nebenwirkungen.

```

1  import java.util.ArrayList;
2  import java.util.List;
3
4  public class FakeLogger implements Logger {
5
6      private List<String> messages = new ArrayList<>();
7
8      public List<String> getMessages() {
9          return messages;
10     }
11
12     @Override
13     public void info(String message) {
14         messages.add("INFO: " + message);
15     }
16
17     @Override
18     public void warn(String message) {
19         messages.add("WARN: " + message);
20     }
21
22     @Override
23     public void error(String message) {
24         messages.add("ERROR: " + message);
25     }
26 }

```

In einem Unit-Test könnte man diesen FakeLogger wie folgt einsetzen:

```

1  public class OrderServiceTest {
2
3      public static void main(String[] args) {
4          FakeLogger fakeLogger = new FakeLogger();
5          OrderService service = new OrderService(fakeLogger);
6
7          service.placeOrder("ABC-123", -1);
8
9          // Ausgabe der gesammelten Log-Meldungen zu Testzwecken
10         for (String msg : fakeLogger.getMessages()) {
11             System.out.println(msg);
12         }
13     }
14 }

```

In echten Testframeworks (z.B. JUnit) würde man an dieser Stelle nicht nur auf die Konsole ausgeben, sondern mit Assertions prüfen, ob die erwarteten Meldungen im FakeLogger enthalten sind (z.B. ob bei einer ungültigen Menge eine Warnung geloggt wurde).

10 UML-Klassendiagramm des Logger-Beispiels

Abbildung 2 zeigt das UML-Klassendiagramm für das Beispiel mit dem Interface `Logger`, den konkreten Logger-Implementierungen und der `OrderService`-Klasse.

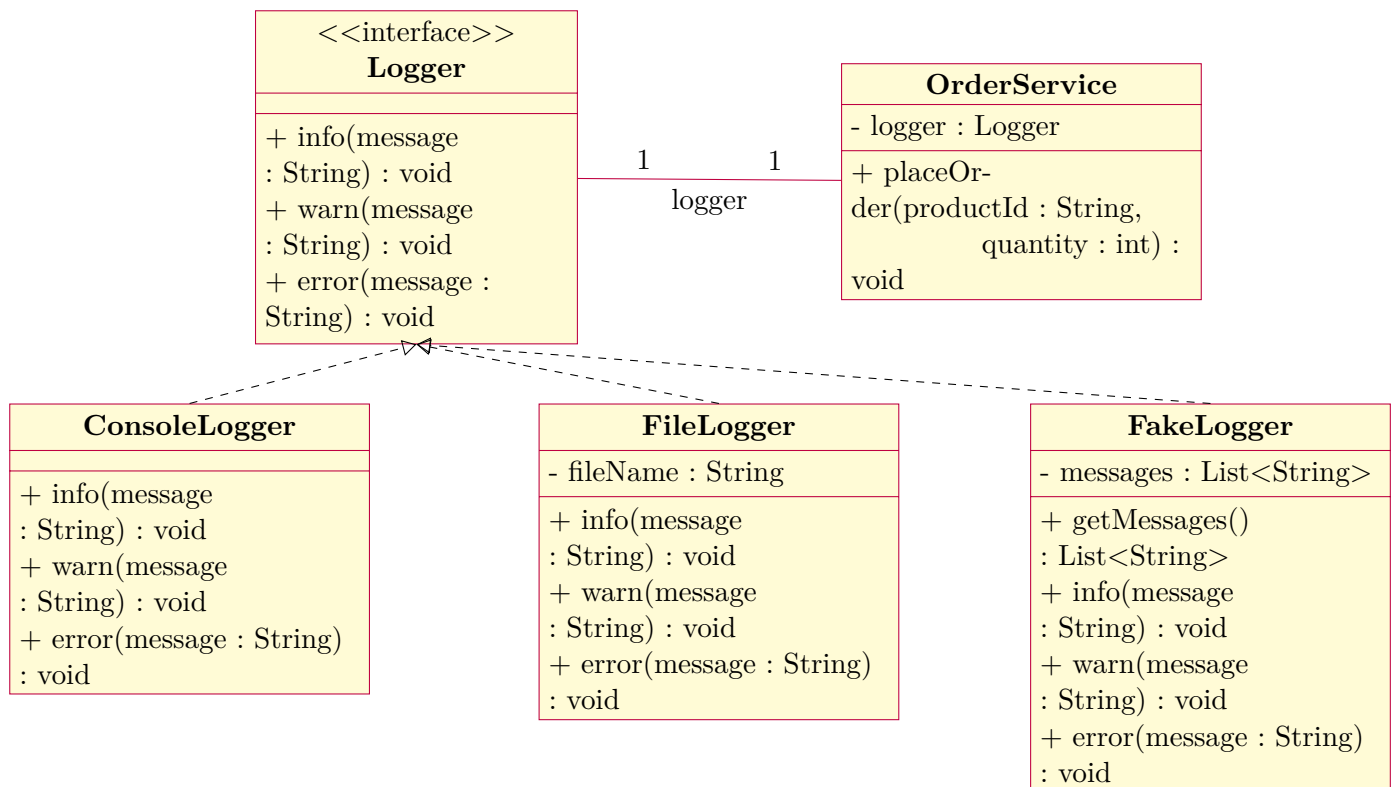


Abbildung 2: UML-Klassendiagramm für das Logger-Beispiel mit Interface `Logger`.

11 Zusammenfassung

- Das **Interface** `PaymentService` definiert, *was* eine Zahlungsart können muss (Methode `pay(double amount)`), die konkreten Klassen entscheiden, *wie* die Zahlung technisch umgesetzt wird.
- Durch die Validierung von `amount` werden unsinnige Beträge abgefangen; der `Shop` arbeitet nur mit dem Interface und bleibt so flexibel und gut testbar (z. B. mit `FakePaymentService`).
- Das Interface `Logger` beschreibt allgemein die Fähigkeit zu loggen (`info`, `warn`, `error`), unabhängig davon, ob auf Konsole, in eine Datei oder anderswo geschrieben wird.
- Konkrete Logger wie `ConsoleLogger`, `FileLogger` und `FakeLogger` zeigen, wie sich Implementierungen leicht austauschen lassen (Betrieb vs. Tests), ohne den Anwendungscode (z. B. `OrderService`) zu ändern.
- Beide Beispiele verdeutlichen das Prinzip „**Programmiere gegen ein Interface, nicht gegen eine Implementierung**“ und zeigen, wie Interfaces zu Entkopplung, Erweiterbarkeit und Testbarkeit beitragen.

12 Anhang: Prinzip „Programmiere gegen ein Interface, nicht gegen eine Implementierung“

Das Prinzip „Programmiere gegen ein Interface, nicht gegen eine Implementierung“ bedeutet im Kern:

Ihr Code soll sich auf **Abstraktionen** (Interfaces/Basistypen) verlassen, nicht auf **konkrete Klassen**.

Im Folgenden wird dieses Prinzip an den beiden Beispielen `PaymentService` und `Logger` verdeutlicht.

1. Was heißt „gegen ein Interface programmieren“?

Betrachten wir zwei Varianten für die Klasse `Shop`.

Variante A – gegen die Implementierung programmieren

```
1 public class Shop {
2
3     private final CreditCardPaymentService paymentService;
4
5     public Shop() {
6         this.paymentService = new CreditCardPaymentService("1234...");
7     }
8
9     public void checkout(double amount) {
10        paymentService.pay(amount);
11    }
12 }
```

- Der Shop kennt explizit die konkrete Klasse `CreditCardPaymentService`.
- Wenn später PayPal oder Rechnung eingeführt werden soll, muss der Code von `Shop` angepasst werden.
- In Tests lässt sich `Shop` nur schwer isoliert prüfen, weil er sich seine Zahlungsart selbst erzeugt.

Variante B – gegen das Interface programmieren

```
1 public class Shop {
2
3     private final PaymentService paymentService;
4
5     public Shop(PaymentService paymentService) {
6         this.paymentService = paymentService;
7     }
8
9     public void checkout(double amount) {
10        paymentService.pay(amount);
11    }
12 }
```

- `Shop` kennt nur das Interface `PaymentService`.
- Welche konkrete Klasse dahintersteckt (`CreditCardPaymentService`, `PaypalPaymentService`, `InvoicePaymentService`, `FakePaymentService`) ist dem `Shop` egal.

- Die Entscheidung, *welche* Implementierung genutzt wird, trifft man außerhalb von **Shop** (z. B. in **Main**, über Konfiguration oder Dependency Injection).

Das ist „gegen das Interface programmieren“: Der Code sagt im Grunde nur: „Ich brauche etwas, das **PaymentService** erfüllt“, nicht: „Ich brauche genau diese Klasse“.

2. Warum ist das so mächtig? (Payment-Beispiel)

Austauschbarkeit

```
1 PaymentService payment = new CreditCardPaymentService("...");
2 // spaeter:
3 payment = new PaypalPaymentService("...");
```

- Der Code von **Shop** bleibt unverändert.
- Nur die konkrete Implementierung von **PaymentService** wird ausgetauscht.

Erweiterbarkeit Neue Zahlungsarten können einfach als weitere Klassen eingeführt werden, die **PaymentService** implementieren. Alle Stellen, die nur den Typ **PaymentService** kennen, funktionieren weiterhin, ohne dass sie angepasst werden müssen.

Testbarkeit Im Test kann ein **FakePaymentService** verwendet werden:

```
1 PaymentService fake = new FakePaymentService(true);
2 Shop shop = new Shop(fake);
```

- Es findet keine echte Zahlung statt.
- Es werden keine externen Systeme angesprochen.
- Getestet wird nur das Verhalten von **Shop**, wenn **pay true** oder **false** zurückliefert.

3. Dasselbe Prinzip beim Logger-Beispiel

Schlechte Variante (gegen Implementierung programmieren)

```
1 public class OrderService {
2
3     private final ConsoleLogger logger = new ConsoleLogger();
4
5     public void placeOrder(...) {
6         logger.info("...");
7     }
8 }
```

- **OrderService** ist fest an **ConsoleLogger** gebunden.
- Wenn in eine Datei geloggt werden soll, muss **OrderService** geändert werden.

Gute Variante (gegen Interface programmieren)

```
1 public class OrderService {
2
3     private final Logger logger;
4
5     public OrderService(Logger logger) {
6         this.logger = logger;
7     }
8
9     public void placeOrder(...) {
10         logger.info("...");
11     }
12 }
```

- OrderService kennt nur den Typ Logger.
- Ob ein ConsoleLogger, FileLogger oder FakeLogger verwendet wird, ist austauschbar.
- OrderService bleibt unverändert, auch wenn die Logging-Strategie später geändert wird.

Im Test kann beispielsweise ein FakeLogger genutzt werden:

```
1 FakeLogger fakeLogger = new FakeLogger();
2 OrderService service = new OrderService(fakeLogger);
3 service.placeOrder("ABC-123", -1);
4 // Nun kann im FakeLogger geprüft werden, ob die erwartete Warnung
   geloggt wurde.
```

4. Der allgemeine Kern in einem Satz

- Ohne Interfaces hängen Klassen oft direkt aneinander (z. B. OrderService braucht genau ConsoleLogger).
- Mit Interfaces kennen Klassen nur noch Verträge (z. B. OrderService braucht „irgendetwas, das wie ein Logger funktioniert“).

Daraus ergeben sich wichtige Vorteile:

- **Entkopplung:** Weniger direkte Abhängigkeiten zwischen Klassen.
- **Flexibilität:** Neue Varianten können leicht eingeführt werden.
- **Testbarkeit:** Fakes oder Mocks können eingesetzt werden.
- **Bessere Architektur:** Schichten und Komponenten kommunizieren über klar definierte Interfaces.

Merksatz: Programmieren gegen Interfaces

Merksatz:

Statt Klassen direkt von konkreten Klassen abhängig zu machen, sollte der Code immer nur von **Interfaces** bzw. abstrakten Typen abhängen.

Programmiere gegen ein Interface, nicht gegen eine Implementierung: Klassen wissen nur, *welches Verhalten* sie benötigen (z.B. PaymentService, Logger), aber nicht, *wie* dieses Verhalten technisch umgesetzt ist. So bleiben Anwendungen **flexibel**, **erweiterbar** und **gut testbar**.